

DESARROLLO DE APLICACIONES Y SERVICIOS

Universidad Pontificia Comillas (ICAI)

Práctica 3: React y NextJS

Documentación del Sprint 3

Autores:

Jorge Carnicero Príncipe

Andrés Gil Vicente

GitHub: <https://github.com/jorgecarnicero/proyecto-final-das.git>

Índice

1. Tareas a realizar y Organización de la carga	2
2. Dificultades encontradas	3
3. Desviaciones producidas	4
4. Ampliaciones Implementadas	5
4.1. Cambio de Tema	5
4.2. Filtrado por contenido y nombre	7
5. Resultados	10

1. Tareas a realizar y Organización de la carga

En este apartado exponemos las tareas que nos hemos propuesto para este tercer Sprint, registradas como *issues* en nuestro repositorio, así como el reparto que hemos llevado a cabo:

- **Issue 1: Configuración Inicial de NextJS y Layout Base (Responsable: Jorge C. P.).** El primer paso del sprint fue pasar el proyecto de HTML a NextJS. Se organizó la estructura de carpetas siguiendo las convenciones del framework (`app/`, `components/`, `styles/...`) dentro de la carpeta de frontend. También se implementó el layout global que comparten todas las páginas, formado por un *Header* con el nombre de la aplicación y la navegación principal, un *main* con el contenido principal de cada una de las páginas que será el que ira cambiando, y un *Footer* con información básica. Usamos este layout para que actúe como envoltorio común para mantener una apariencia coherente en toda la aplicación.
- **Issue 2: Autenticación — Pantallas de Registro y Login en React (Responsable: Andrés G. V.).** Se reescribieron las páginas de registro y login que ya teníamos en HTML, transformándolas en componentes. Ambas pantallas gestionan su propio estado con el hook `useState` y aplican las mismas validaciones que ya teníamos en la práctica anterior: comprobación del formato del email y, en el registro, que las dos contraseñas introducidas coincidan entre ellas y cumplan los requisitos mínimos. Al completar el login con éxito, el token de sesión devuelto por el backend se almacena en el *LocalStorage* para que el resto de la aplicación pueda autenticar sus peticiones.
- **Issue 3: Interfaz de Chat Activo e Interacción con el LLM (Responsable: Jorge C. P.).** Esta ha sido una de las tarea más complejas del sprint. Hemos creado la pantalla principal del chat, donde el usuario puede escribir mensajes y el LLM le responderá. Los mensajes del usuario se muestran alineados a la derecha y los del LLM a la izquierda, tal y como se nos pedía en el enunciado. También hemos añadido una pantalla intermedia donde se mostrará una pequeña animación en el mensaje que mande el LLM para que sea más visual para el usuario y que sepa que no ha fallado nada, sino que el LLM esta pensando qué responder. Además, validamos también que no se pueda enviar un mensaje vacío.
- **Issue 4: Panel Inicial e Historial de Chats (Responsable: Andrés G. V.).** Con esta tarea hemos desarrollado la pantalla de inicio a la que accede el usuario tras hacer login, donde puede ver todos sus chats anteriores, tokens, filtrar, etc. Desde esta vista también se puede seleccionar un chat existente para seguir una conversación, crear un chat nuevo o eliminar cualquier chat del historial. La lista de chats la cargamos dinámicamente al entrar en la página mediante una petición al backend autenticada con el token del usuario.
- **Issue 5: Perfil de Usuario y Ajustes de Cuenta (Responsable: Jorge C. P.).** Implementamos la pantalla de perfil, desde la que el usuario puede consultar su información personal y modificar su nombre de usuario o contraseña. El formulario de

cambio de contraseña incluye validaciones: comprobamos que la nueva contraseña y su confirmación coincidan. Estos cambios, cuando los haya, se envían al backend mediante las peticiones correspondientes a los endpoints del perfil.

- **Issue 6: Pruebas con Docker y Documentación del Sprint (Responsabilidad conjunta).** Antes de cerrar el sprint, hemos verificado que toda la aplicación arranca correctamente con el comando de `docker compose up`, comprobando que el frontend, el backend y ollama se levantan sin errores y que los servicios se comunican entre sí, por ejemplo viendo que el chatbot nos responde a los mensajes y que los registros o cambios de contraseñas, etc. funcionan correctamente. También hemos revisado que no nos saliera ningún *warning* en la consola del navegador antes de hacer la entrega final. Por último, hemos redactado esta memoria final documentando el trabajo que hemos realizado durante todo el sprint, mostrando el paso a paso, dificultades y las decisiones que hemos ido tomando.

2. Dificultades encontradas

En general, este sprint ha sido el más difícil de los 3 que hemos hecho, ya que teníamos que combinar tanto frontend como backend, aunque la parte del backend era más secundaria ya que ya estaba terminada pero era necesario saber cómo funcionaba correctamente para poder hacer bien las peticiones que se necesitaban en este último sprint. A continuación, detallamos esto un poco más:

- **Migración de HTML a React y NextJS:** Al principio nos costó entender bien cómo funciona el sistema de rutas de NextJS con el App Router, sobre todo en lo que respecta a los layouts anidados y a cuándo usar componentes de servidor frente a componentes de cliente. Tuvimos que dedicar un rato a entender cuándo era necesario añadir `'use client'` al principio de un componente para no encontrarnos con errores inesperados. Ya que nos pasaba que intentábamos usar los hooks vistos en clase y nos daba error al no tener puesto el `'use client'` y no sabíamos si era porque nuestra lógica estaba mal.
- **Cambio de Nombre de Usuario:** Sin duda, una de las mayores dificultades que hemos tenido, ha sido cuando ya teníamos la app de profile lista, y funcionando de forma que el cambio de contraseña nos funcionaba bien. Nos dimos cuenta de que cambiar el nombre de usuario no daba error pero no funcionaba como esperábamos, tras cambiar el nombre varias veces e intentar iniciar sesión, no identificaba ese usuario (el nuevo). De forma que echando la vista atrás y mirando en el backend nos dimos cuenta de que en el serializador teníamos marcado el campo de username como un campo de solo lectura (read only) de forma que no podíamos modificarlo. Una vez descubierto esto tuvimos que apañarnoslas para cambiar el cómo se manejaba, logrando que finalmente funcionara bien el cambio de nombre de usuario en el apartado del perfil.

- **Configuración de Docker con el frontend de NextJS:** Integrar el frontend de NextJS en el `docker-compose` existente presentó más retos de lo esperado. Aunque no fue necesario modificar el `Dockerfile`, tuvimos que adaptar el script de arranque en el `package.json` para asegurarnos de que el servidor de producción de NextJS fuera accesible correctamente desde fuera del contenedor y que las variables de entorno llegaran adecuadamente.
- **Extra de filtrado:** Una pequeña dificultad en el filtrado, fue que al filtrar tanto en si el texto a buscar puede estar en el contenido de los mensajes del LLM o en el nombre del Chat, manejar el filtrado de varios condicionales dentro de un `.filter()` nos lio un poco. Para usar esas condiciones lógicas de OR, etc. tuvimos que consultar alguna documentación.

3. Desviaciones producidas

- **Issue 3 más costoso de lo previsto:** La interfaz de chat llevó más tiempo del estimado porque, además de la lógica de mensajes, tuvimos que gestionar los estados de carga y error, que al principio no habíamos tenido en cuenta en la planificación. Esto hizo que terminar este issue nos costara algo más de lo que inicialmente habíamos pensado.
- **Más iteraciones de las esperadas en el perfil de usuario:** La validación del formulario de cambio de contraseña nos requirió varias revisiones y cambios, ya que para poder comprobar que funcionaba bien, teníamos que crear un usuario, cambiar la contraseña, ver que funcionaba, cambiar el usuario, ver que funcionaba, si no funcionaba tocar la lógica, y esto repetidas veces hasta que al fin nos fuera todo cómo queríamos.
- **Local Vs Docker:** Otra pequeña desviación fue que al principio estábamos probando únicamente con el frontend desplegado sin tener el backend desplegado con el `docker-compose up`, de forma que las peticiones que hacíamos el backend obviamente iban a ir vacías, ya que no había sitio de fondo como tal. Esta desviación fue pequeña pero aun así nos dio bastantes problemas en su momento.
- **Rutas y parametros del backend:** Otra cosa que nos hizo desviarnos un poco y que tardamos más de lo previsto, era que cuando teníamos que hacer peticiones al backend tanto de enviar información como de recibir información teníamos que saber la ruta y los parametros de la solicitud. De forma que teníamos que tener todo desplegado para poder meternos al Swagger e ir viendo como era el nombre de los parametros y las url para que no dieran fallos.
- A pesar de estas desviaciones, al final hemos podido completar todas las *issues* del milestone del Sprint 3 que nos habíamos propuesto y la aplicación arranca y funciona correctamente, permitiendo desde crear usuarios, iniciar sesión, crear y borrar chats, además de abrir los ya creados, modificar usuario y contraseña, y por supuesto interactuar fluidamente con el LLM mientras tengamos suficientes mensajes disponibles.

4. Ampliaciones Implementadas

4.1. Cambio de Tema

Hemos implementado la funcionalidad extra de Cambio de Tema, permitiendo cambiar entre un modo oscuro y un modo claro. Al principio nuestra idea era crear 2 ficheros css por separado, uno para el modo oscuro y otro para el modo claro, y ya dentro de los archivos correspondientes importar uno u otro en función de un ternario con el contexto del tema.

Finalmente, llegamos a la conclusión de que iba a ser mucho más fácil usar variables de estilo, tanto para la creación como para si luego quisieramos cambiar algo del estilo en general. Entonces lo que hacemos es que en función del contexto que haya, se usarán unas variables u otras, y de esta forma no tenemos que crear 2 css distintos y lo tenemos todo en un menos líneas de código (menos redundancia) sin tener que repetir contenido, que sería también menos claro.

Para cambiar el tema creamos una función que es **toggleTheme**, la cual nos actualiza el estado con una variable intermedia intercambiándonos en función de cual fuera el anterior, esta la pasaremos con el Provider junto con el theme, para poder acceder a ella desde el Header y ejecutarla en el onClick.

Veamos una comparación de cómo queda la interfaz con el modo oscuro y con el modo claro, apreciándose claramente que ambas permiten una buena experiencia de usuario sin causar ninguna molestia:

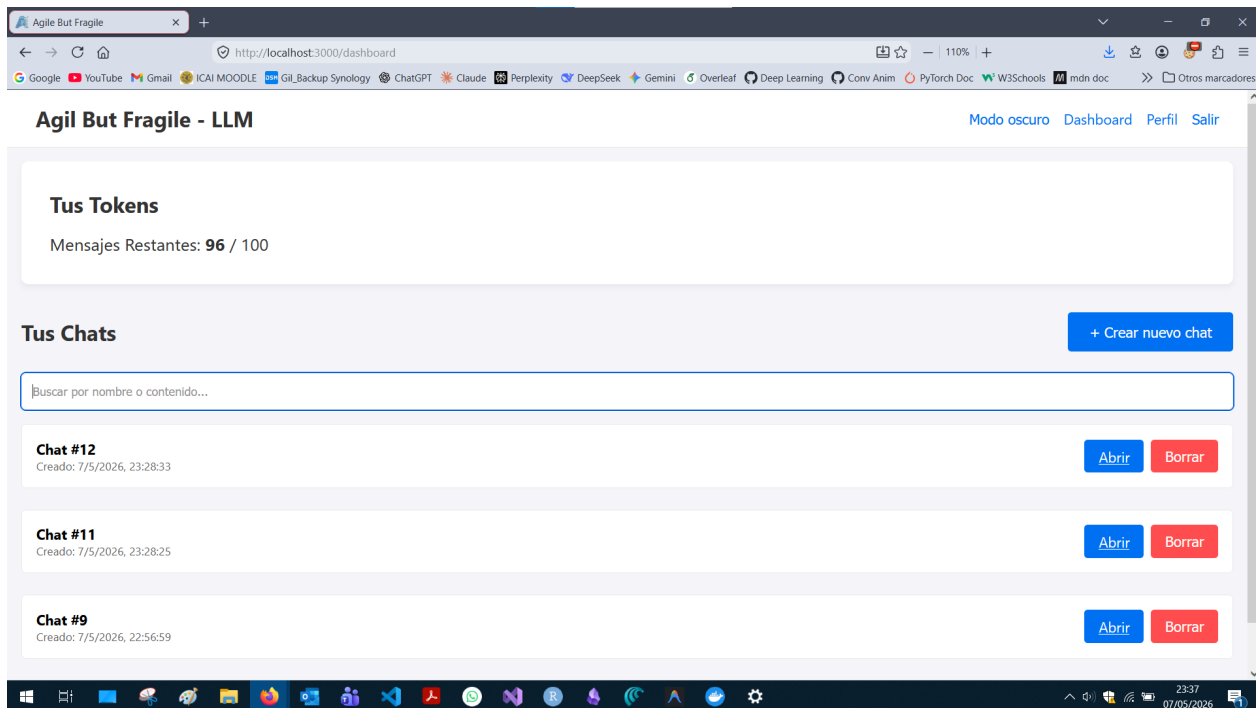


Figura 1: modo claro dashboard

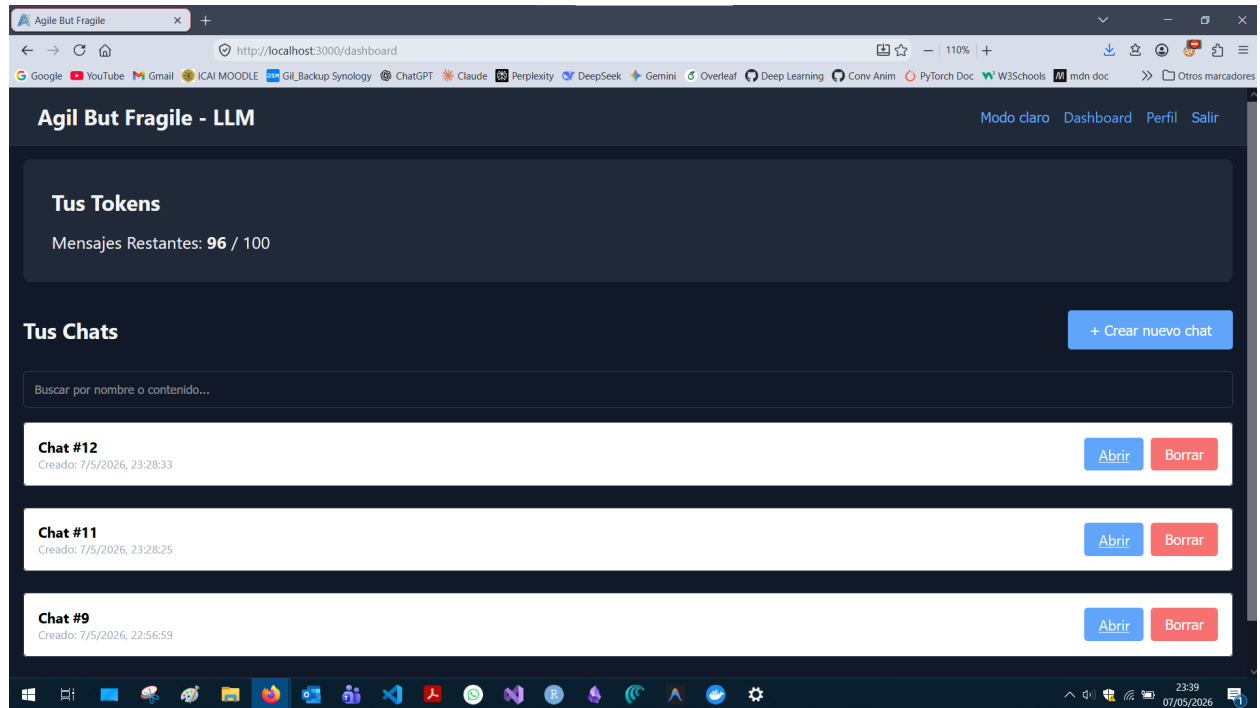


Figura 2: modo oscuro dashboard

4.2. Filtrado por contenido y nombre

También hemos implementado la funcionalidad adicional de Filtrado y Búsqueda en Chats. Para ello, hemos creado una barra de búsqueda que permite al usuario escribir cualquier texto, de modo que solo se le mostrará por pantalla aquellas conversaciones que tengan ese texto en el nombre o como contenido dentro del chat (en algún mensaje de los que se ha intercambiado).

Para comenzar con esta funcionalidad, lo primero que tenemos es un estado que contiene todos los chats del usuario. Al principio lo inicializamos vacío y más tarde cargaremos con un `useEffect()` todos los chats del usuario. Una vez los tenemos crearemos una variable que tendrá todos los chats filtrados en función de lo que se haya puesto como contenido a buscar.

Este filtrado lo hacemos con un `.filter()` como hemos visto en clase, en donde tendremos todos los chats si no hay texto puesto para filtrar, y luego en el momento en el que haya texto para filtrar miraremos si este texto está presente tanto en el nombre del chat como en el contenido generado por el LLM. En caso de estar en alguno de los 2 (usamos condición de OR), entonces dejaremos pasar este chat por filtro para mostrarlo en pantalla.

Veamos una demostración de los resultados que se obtienen al usar esta nueva funcionalidad de filtro:

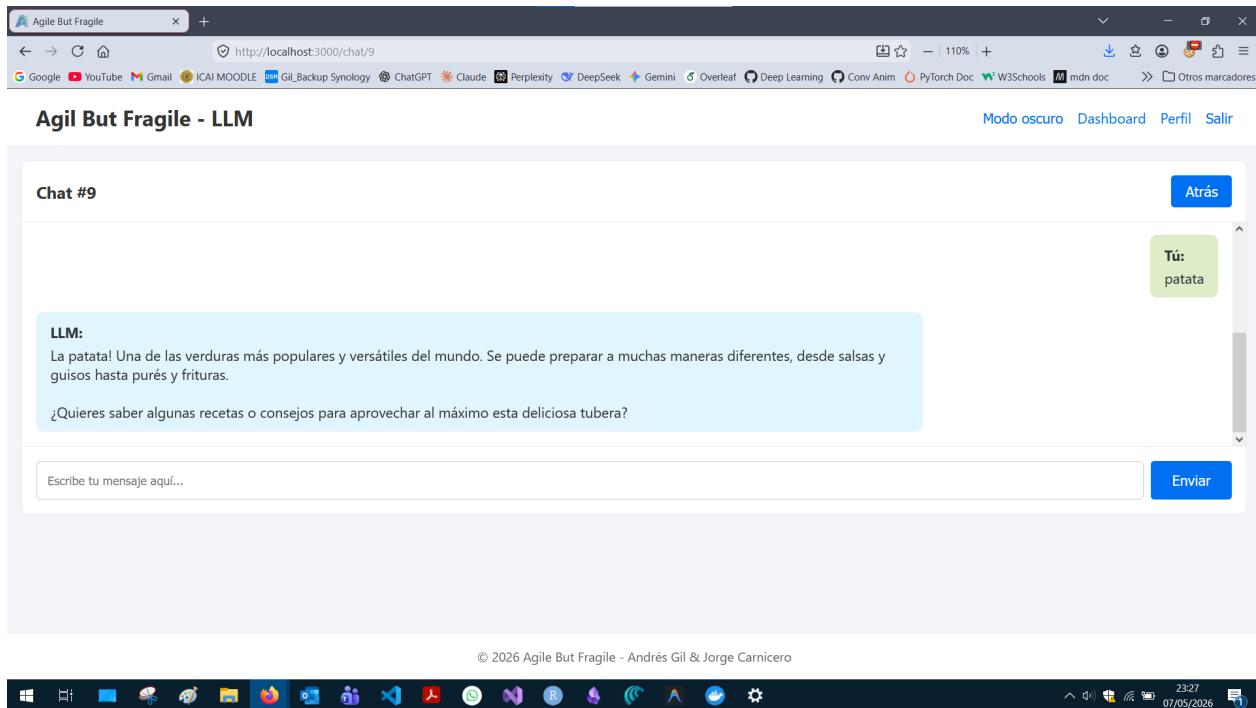


Figura 3: ejemplo chat con palabra "patata"

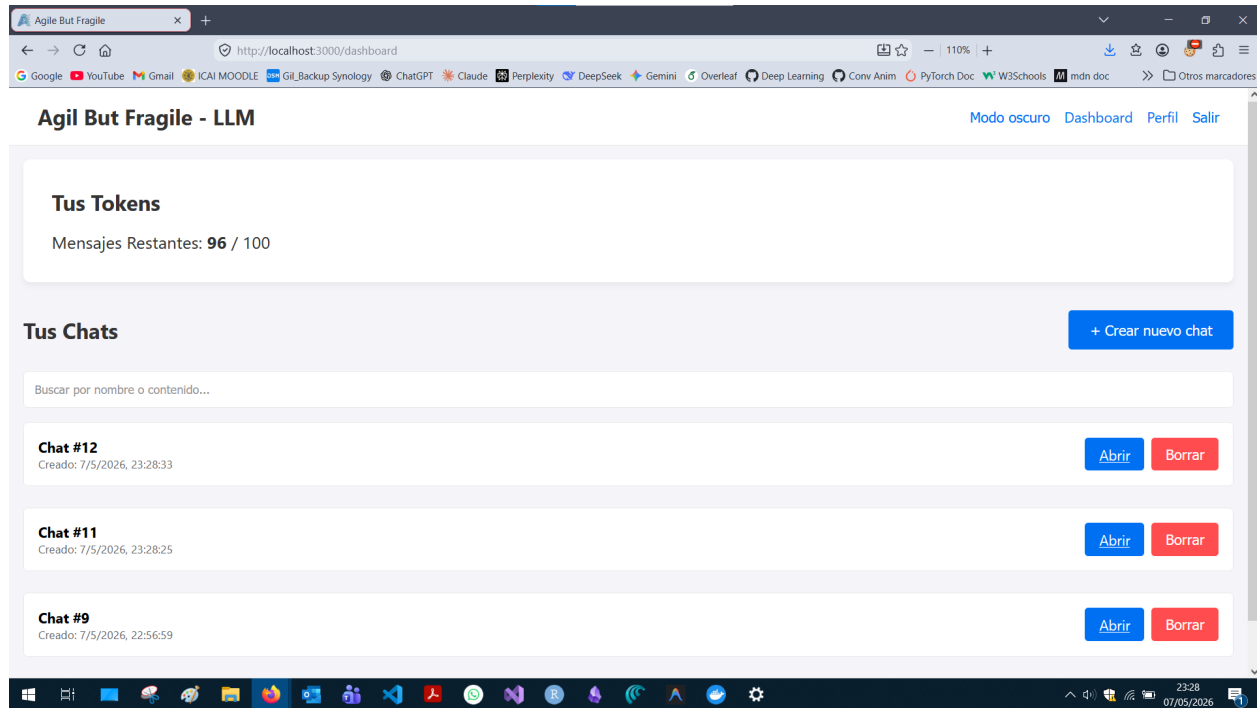


Figura 4: distintos chats existentes

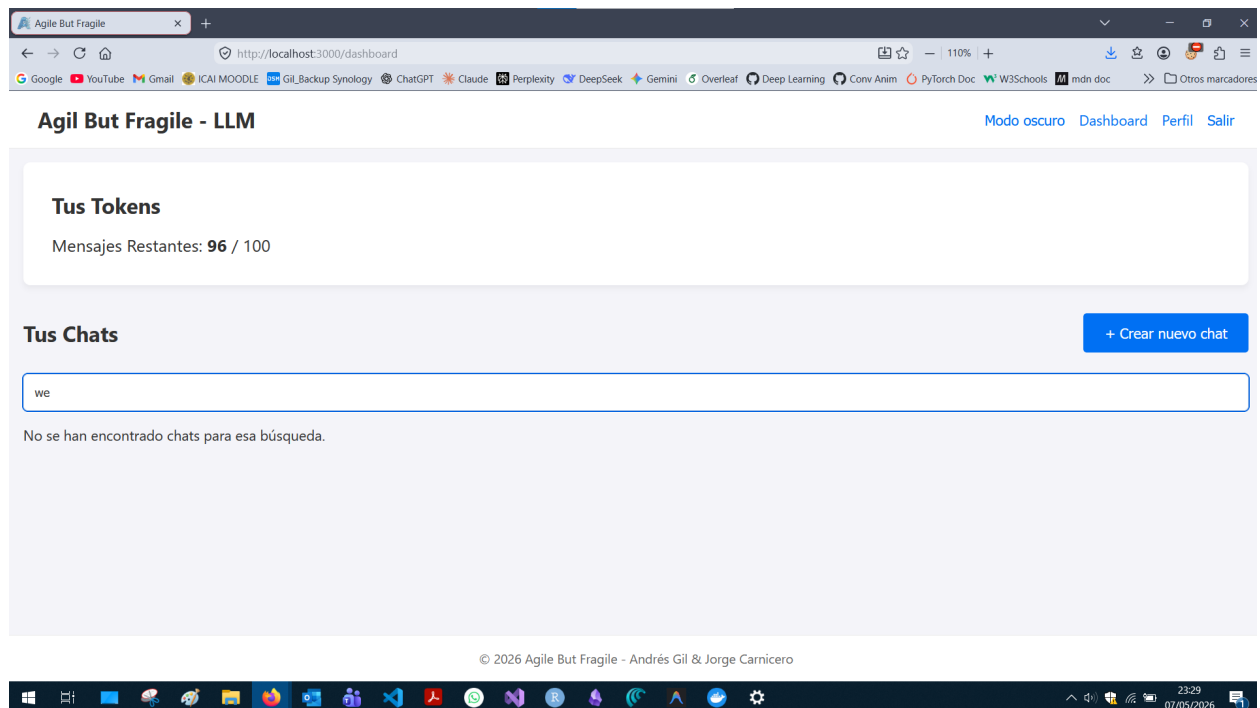


Figura 5: sin resultados de búsqueda para "we"

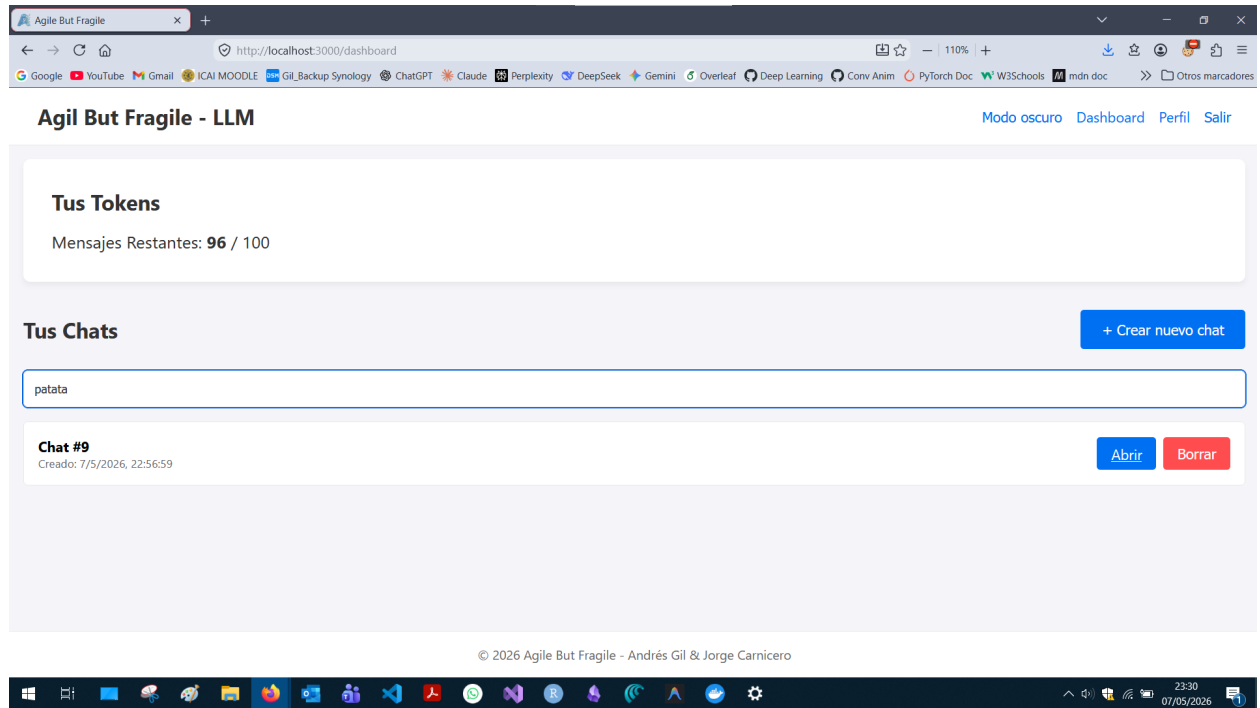


Figura 6: con resultados de búsqueda para "patata"

5. Resultados

A continuación, queremos mostrar algunos resultados visuales de cómo queda nuestra aplicación finalmente, con algunas de sus funcionalidades principales, la interfaz que hemos diseñado, los distintos temas, las distintas pantallas, las respuestas del LLM, etc.

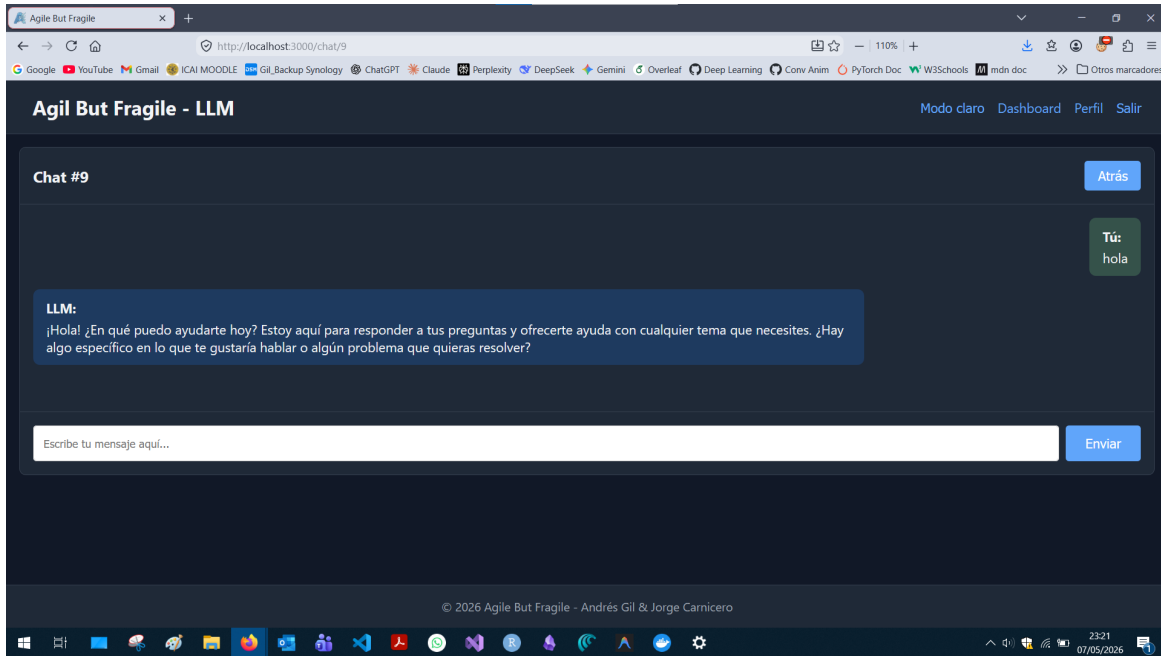


Figura 7: login page

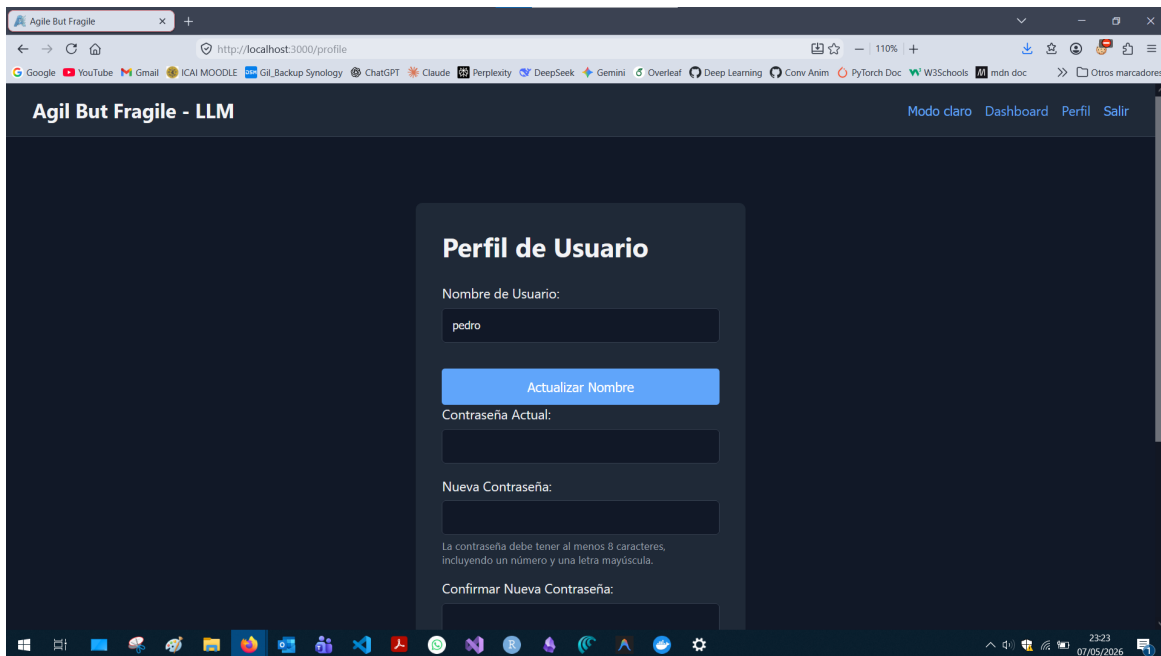


Figura 8: profile page

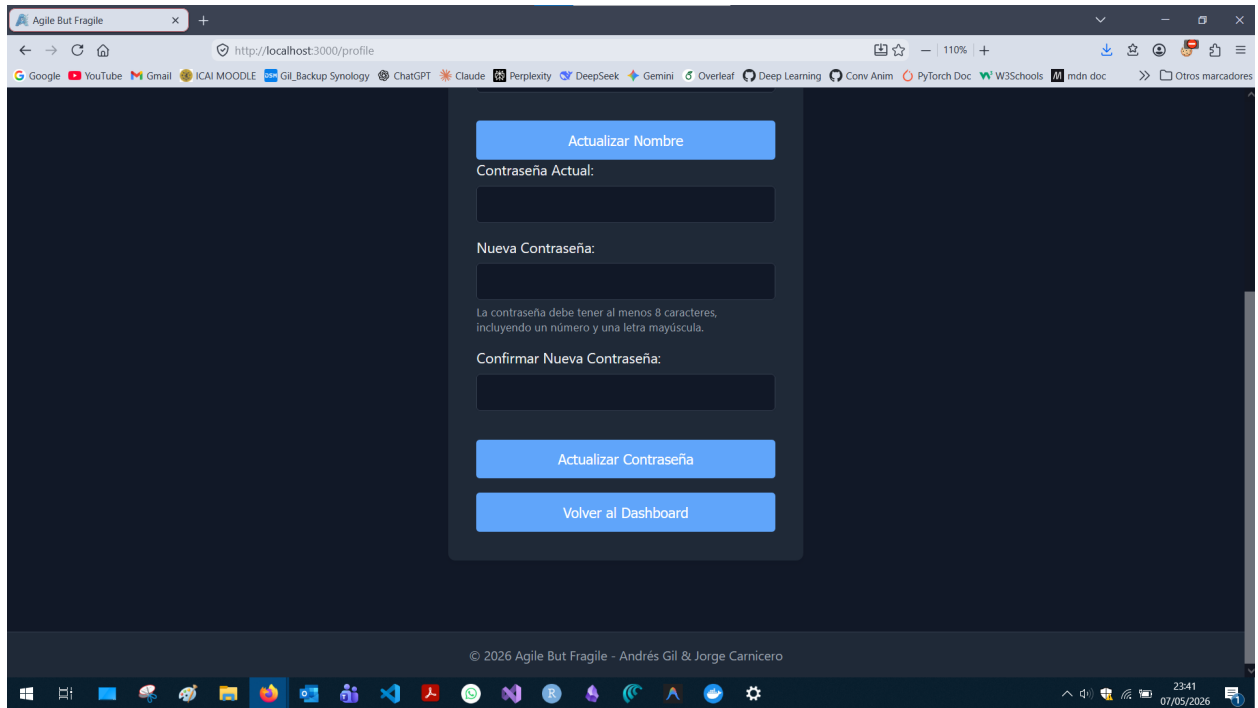


Figura 9: actualización de credenciales en el perfil

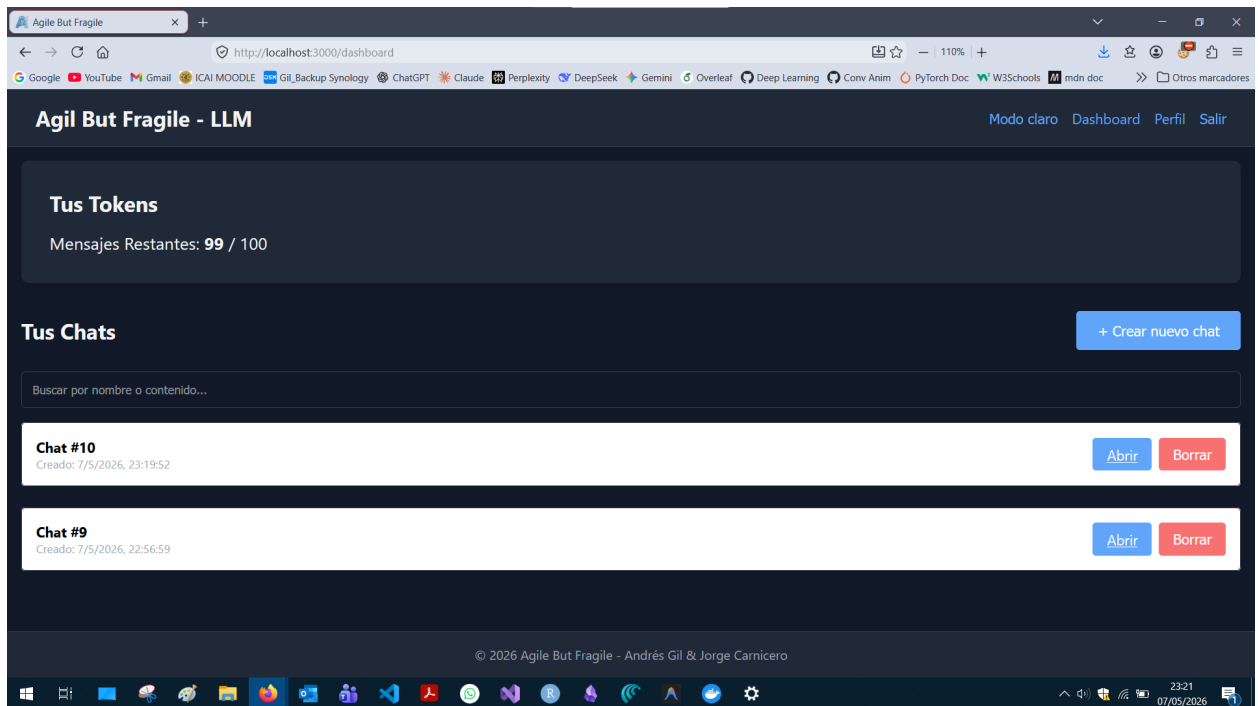


Figura 10: dashboard page

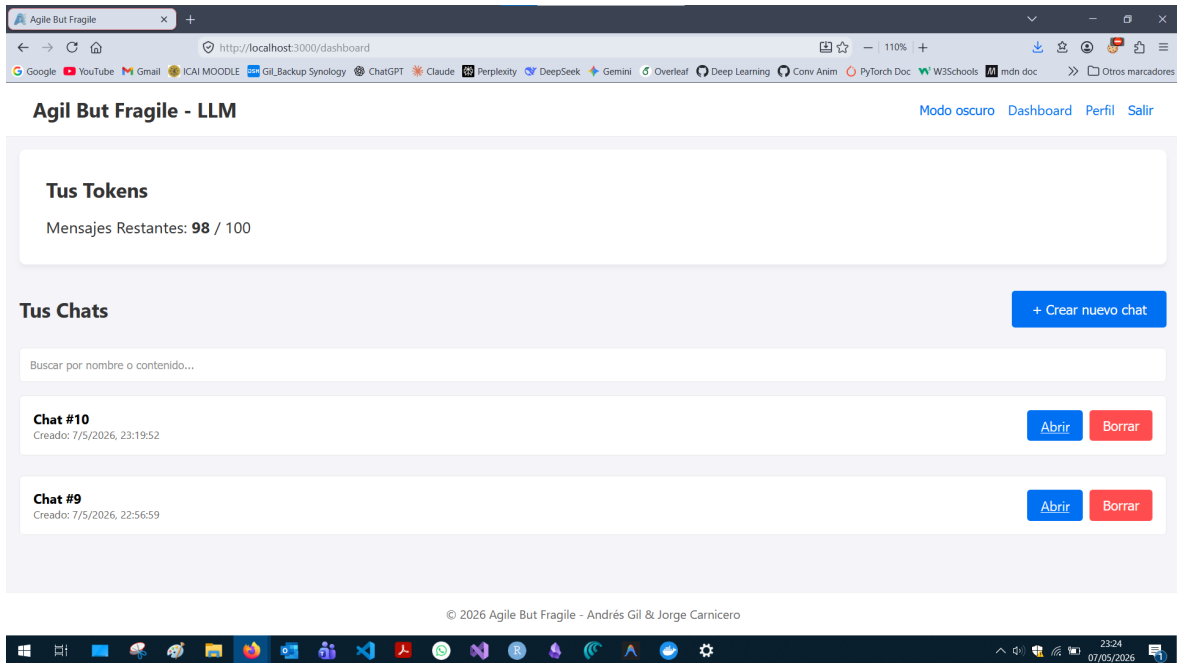


Figura 11: dashboard page modo claro

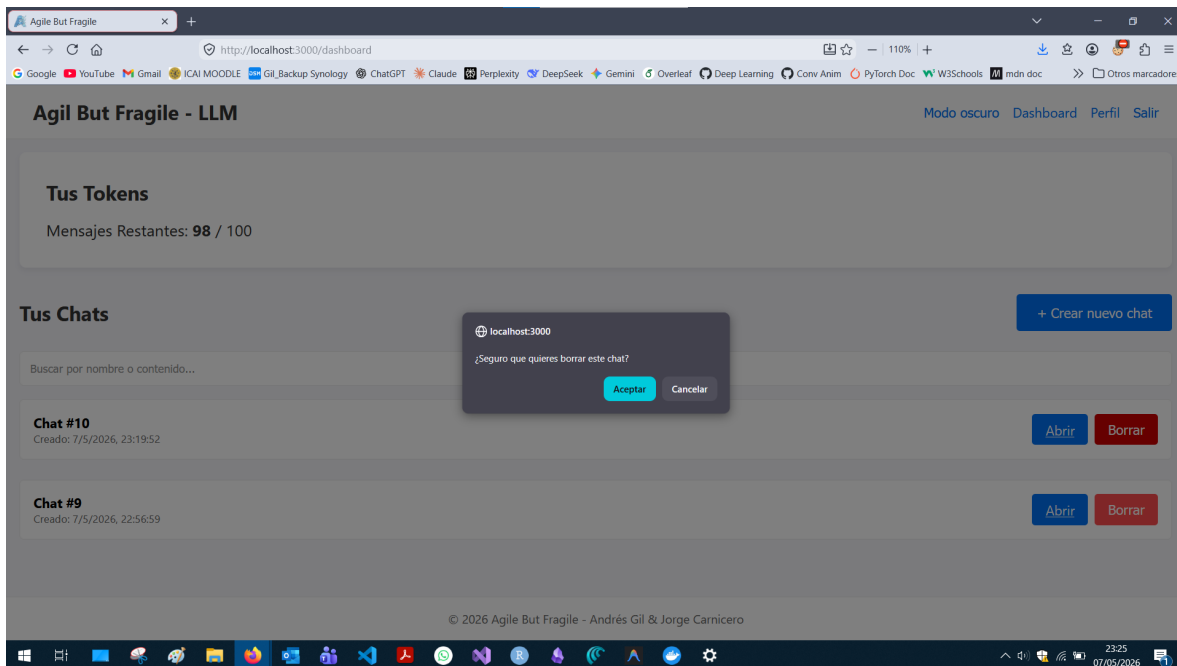


Figura 12: opción de borrado de chat

Todas estas imágenes muestran que la aplicación funciona correctamente y que se despliega sin problemas cuando levantamos el docker, pudiéndose ver en la URL del navegador que estamos abriendo la aplicación en la ruta que se especifica en el README del proyecto.